Building an EV battery health certificate app with the Claude API

Option C — a static JSON spec database, free government APIs, and Claude API only for narrative text — is the right architecture for AZ Autos. This approach gives you verified specs on every certificate, costs under £0.10 per certificate, and eliminates the hallucination risk that makes the pure-Claude approach unsuitable for customer-facing documents. The DVLA VES API (free) confirms the vehicle is real and electric, the MOT History API (free) supplies the model name and mileage history, your own JSON file holds verified EV specs for the ~20 models you'll encounter, and Claude generates only the plain-English condition summary, range explanation, and warranty narrative. This report covers every technical detail needed to build it.

Claude knows EV specs — but not reliably enough for certificates

Claude's training data includes extensive automotive content and it can recall battery capacity, WLTP range, charging speeds, and warranty terms for popular models like the Tesla Model 3 or Nissan Leaf with reasonable accuracy. For well-documented, high-volume EVs, Claude typically gets the headline figures right: **75 kWh usable** for a Model 3 Long Range, (ev-database) **282 miles WLTP** for a Kia e-Niro 64 kWh, **8 years / 100,000 miles** for most battery warranties.

The problem is the edge cases. Claude cannot reliably distinguish between trim-level variants — a Tesla Model 3 Standard Range Plus had ~50 kWh usable (EVSpecifications) with NCA chemistry pre-2021 but ~57.5 kWh usable with LFP chemistry post-2021, and Claude may confuse these. It can mix up gross and usable battery capacity (the Audi e-tron 55 has 95 kWh gross (Electrichasgoneaudi) but only 86.5 kWh usable post-software-update — previously 83.6 kWh usable). (EV Database) (EV Database) Year-by-year spec changes, regional variants, and mid-production-run updates are exactly the kind of precise numerical data where LLMs hallucinate most confidently.

No published research specifically benchmarks LLM accuracy for EV specifications. However, automotive industry practice — from Porsche Engineering to SAE technical papers — consistently treats LLMs as text generators grounded by verified databases, never as standalone knowledge sources for specifications. **A single wrong battery capacity figure on a customer certificate (stating 77 kWh when the car has 64 kWh) would destroy credibility instantly.** The risk-reward calculation is clear: never let Claude supply unverified numbers on a professional document.

For the specific UK models you'll encounter, here are the warranty terms Claude would need to get right — and where mistakes would matter most:

Manufacturer	Warranty	SoH threshold	Key caveat
Tesla (SR/RWD)	8 yr / 100k mi	70%	LR/Performance gets 120k miles
Nissan	8 yr / 100k mi	~70% (9/12 bars)	Air-cooled; no active thermal management
Kia	**7 yr** / 100k mi	70%	Shortest warranty of major OEMs
Hyundai	8 yr / 100k mi	70%	2019-2020 Kona had battery fire recall
VW / Škoda	8 yr / 100k mi	70%	MEB platform shared terms
MG	8 yr / 100k mi	70%	Vehicle warranty shorter (7yr/80k)
BMW	8 yr / 100k mi	70%	Liquid-cooled
Audi	8 yr / 100k mi	70%	22 kW AC optional on some specs
Renault	8 yr / 100k mi	**66%**	Lowest threshold; check battery lease vs owned

Renault's **66% SoH threshold** (not 70%) Beev Clearwatt and Kia's **7-year term** (not 8) Speak EV Kia are precisely the sort of details Claude might get wrong in a way that misleads a buyer about remaining warranty coverage.

Three architectures compared — and why Option C wins

Option A (Pure Claude API) is the simplest to build: pass the registration, model, year, and SoH to Claude and let it return everything. Cost is roughly **£0.03–0.04 per certificate**. But there is no data validation, no official vehicle identity confirmation, and every specification number carries hallucination risk. This is unsuitable for any document a customer takes away.

Option B (DVLA API + Claude API) is better: the DVLA confirms the vehicle exists and is electric, the MOT API provides the model name and mileage history, then Claude fills in EV specs and generates narrative. Cost is similar at **£0.03–0.05 per certificate** since both government APIs are free. The problem remains that Claude is still providing unverified battery capacity, charging speeds, and warranty terms — the most important data on the certificate.

Option C (Static JSON + Government APIs + Claude for narrative only) solves this. Every factual number comes from a verified source. Claude's role is limited to what it does best: generating well-written, contextualised prose. Cost is **£0.03–0.06 per certificate** for models in your database, and the maintenance burden of a 20–30 model JSON file is roughly 2–4 hours to compile initially, then periodic updates when new models appear in your stock.

An enhanced version adds UKVD's EV Data API (£0.15/lookup) ukvehicledata as a fallback for vehicles not in your static database, bringing the cost to **£0.18–0.21** for unknown models only. For a dealer doing 10–50

certificates per month, annual Claude API cost would be **£4–36**, and even with occasional UKVD lookups the total would stay under £100 per year.

Factor	Option A (Pure Claude)	Option B (Hybrid)	Option C (Static DB + Claude narrative)
Spec accuracy	❌ Unverified	⚠️ Unverified EV specs	✅ Manually verified
Vehicle identity	❌ None	✅ DVLA official	✅ DVLA official
Cost per cert	~£0.03	~£0.04	~£0.05
Maintenance	None	Low	Moderate (JSON file)
Customer suitability	❌ Unsuitable	⚠️ Risky	✅ Professional
Claude dependency	Total	High	Narrative only

What the free government APIs actually give you

The **DVLA Vehicle Enquiry Service (VES) API** is genuinely free, returns JSON, and requires only an API key in the x-api-key header. Register at the DVLA developer portal by describing your use case. api For a registration lookup, it returns make, year of manufacture, fuel type (returns "ELECTRICITY" for BEVs), CO2 emissions, tax status, MOT status, and colour. Api Critically, **it does not return the model name** — only the make (e.g., "TESLA" but not "Model 3 Long Range"). It also returns nothing EV-specific: no battery capacity, no range, no charging data.

The **MOT History API** from DVSA fills the model gap. It is also free and returns the model name GOV.UK (e.g., "MODEL 3"), first-used date, and complete MOT test history including **odometer readings** at each test — valuable for correlating mileage progression with battery degradation. The new API (September 2025) uses Microsoft OAuth2 authentication, GOV.UK which is more complex than the DVLA's simple API key, but Claude Code can handle the implementation.

Between these two free APIs, you get: confirmed make + model + year + fuel type + registration date + mileage history. The gap is everything EV-specific: battery capacity, WLTP range, charging types and speeds, and warranty terms. This is exactly what your static JSON database fills.

UKVD (UK Vehicle Data) at **£0.15 per lookup** (Tier 1 pricing, £50 pre-payment) is the best commercial fallback. Their EV Data API returns battery capacity in kWh, chemistry, voltage, warranty months and miles, charge port type and location, max charge power, and WLTP range Oneautoapi — essentially everything you need. Two open-source GitHub repositories (**KilowattApp/open-ev-data** under MIT licence and **open-ev-data/open-ev-data-dataset** under CDLA-Permissive) can seed your static database with verified data.

Prompt engineering that produces reliable, professional output

Anthropic now offers **Structured Outputs** (GA since November 2025) — constrained decoding that provides mathematical guarantees of JSON schema compliance. claude Use output_config.format with a json_schema type to force Claude's entire response into your schema. claude For the certificate app, this replaces fragile prompt-engineering approaches to JSON formatting. The Python SDK supports Pydantic models directly via client.messages.parse(), which returns a typed object: Claude API Docs

```python
from pydantic import BaseModel
from anthropic import Anthropic

class BatteryNarrative(BaseModel):
    condition_summary: str
    range_explanation: str
    warranty_status: str
    confidence: str  # "high", "medium", "low"

client = Anthropic()
response = client.messages.parse(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system=SYSTEM_PROMPT,
    messages=[{"role": "user", "content": formatted_vehicle_data}],
    output_format=BatteryNarrative,
)
```

For the system prompt, Anthropic's documentation recommends XML tags (Claude was trained with them), Anthropic explicit rules, and concrete tone descriptions using analogies rather than abstract adjectives. The most effective system prompt for this use case:

```xml
```

```
<system_prompt>
You are an EV Battery Assessment Specialist writing for a professional
vehicle certification service. Your role is to generate clear, honest
certificate narratives from verified vehicle data provided to you.

<rules>
1. ONLY use data provided in <vehicle_data> tags. Never supplement with
   information from your training data for any numerical values.
2. If data is missing, state: "This specification is not available in our
   verified database."
3. Never invent, estimate, or round specification numbers.
4. Use professional, measured language. No superlatives ("amazing",
   "exceptional") and no alarming language unless SoH is genuinely below
   warranty threshold.
5. Never make guarantees about future battery performance.
6. Never use: "rest assured", "peace of mind", "you'll be pleased to know",
   "it's worth noting that", or similar generic phrases.
</rules>

<tone>
Think: a knowledgeable, friendly mechanic explaining results to a car buyer.
Not a salesperson. Not a scientist. Plain language, short sentences, specific
numbers. Honest about both positives and concerns.
</tone>
</system_prompt>
```

The explicit phrase ban in rule 6 is the single most effective technique for preventing AI-sounding output. Combined with **two diverse few-shot examples** showing the exact style you want (Anthropic) — one for a healthy battery, one for moderate degradation — Claude will produce narratives that read as professionally written rather than generated. Always provide the vehicle data, calculated range, and warranty dates as structured input rather than asking Claude to calculate or recall them.

To build in a **confidence flag**, add a field to your Pydantic schema and instruct Claude in the system prompt: "Rate your confidence as HIGH if all data was provided in vehicle_data tags, MEDIUM if you supplemented any detail from general knowledge, LOW if you are uncertain about any figure." Your app then checks this flag and can display a warning or trigger a manual review.

Range estimation should happen in your code, not in Claude

The formula is straightforward: **Estimated range = WLTP range × (SoH / 100) × derating factor**. This calculation should live in your application code, not in a Claude prompt. Claude can then receive the calculated figure and write a plain-English explanation around it, but the arithmetic itself is too important to delegate to an LLM.

Based on aggregated UK data from ClearWatt, What Car?, RAC, and Fleet News, the recommended derating factors for UK conditions are:

Driving scenario	Derating factor	Resulting range vs WLTP
Default mixed UK driving	**0.85**	85% of WLTP
Urban/city	0.95	95% (regen braking helps)
Motorway at 70 mph	0.75	75% (aerodynamic drag)
Winter UK (0–5°C)	0.75	75% (cold battery + heating)
Winter motorway	0.65	65% (worst realistic case)

For the certificate, present **three range scenarios** calculated in your code: a best case (urban, mild weather, factor 0.95), a typical case (mixed driving, factor 0.85), and a conservative case (motorway, cold weather, factor 0.65). For a Skoda Enyaq iV 80 with 90% SoH and 312-mile WLTP range: typical estimated range = 312 × 0.90 × 0.85 = **239 miles**, conservative = 312 × 0.90 × 0.65 = **183 miles**. These numbers get passed to Claude which wraps them in context: "Under typical UK driving conditions, this vehicle's current battery supports an estimated range of approximately 235–245 miles. In cold weather or sustained motorway driving, expect approximately 180–190 miles."

Cost per certificate and production readiness

Claude Sonnet 4.6 costs **$3 per million input tokens** and **$15 per million output tokens**. Anthropic A typical certificate generation requires 2–3 API calls: one for the condition narrative, one for the range explanation, and one for the warranty summary (or combine them into a single structured output call). Each call uses roughly 500–800 input tokens (system prompt + vehicle data) and 200–400 output tokens (narrative text).

With prompt caching enabled — which stores your system prompt at a 90% discount on subsequent reads claude Anthropic — the per-certificate cost breaks down to approximately **£0.03–0.05** ($0.04–0.06). At 50 certificates per month, that is **£1.50–2.50 per month** for Claude API costs. Even at the high end with UKVD fallback lookups, total API costs would stay under **£10 per month**.

Cost component	Per certificate	Monthly (50 certs)	Annual
DVLA VES API	Free	Free	Free
MOT History API	Free	Free	Free
Claude API (narrative)	£0.03–0.05	£1.50–2.50	£18–30
UKVD (fallback only)	£0.15 (if needed)	£0–7.50	£0–90
Total	**£0.03–0.20**	**£1.50–10**	**£18–120**

For reliability, Anthropic offers no formal SLA on its standard API tier — only "best-effort" availability. The **Priority Tier** (requires committed spend) targets 99.5% uptime. Anthropic There was a notable multi-day outage on **2–3 March 2026** affecting all Claude services. STI2 AI News For a small dealer generating certificates during business hours, occasional API downtime is manageable: build a retry mechanism with exponential backoff, and design the app so that if Claude is unavailable, it can still generate a certificate with all the factual data but without the narrative sections (leaving those blank or using a simple template fallback). At 10–50 certificates per month, this is not a critical real-time system.

The recommended tech stack for building with Claude Code is **Next.js deployed on Vercel** — free tier includes 100 GB bandwidth and serverless functions, the EV specs database is simply a JSON file in your repository, and PDF generation uses a library like `@react-pdf/renderer`. The entire app can live in a single repository with no external database server.

Conclusion

Build Option C: static JSON database + free government APIs + Claude for narrative only. The implementation path is clear — register for DVLA VES and MOT History API access (both free, takes a few days), compile your EV specs JSON file using the open-source datasets on GitHub as a starting point (verify against manufacturer data), then build the Claude integration using Structured Outputs with Pydantic schemas. The total API cost is negligible. The architecture cleanly separates verified facts from generated prose, which is the only way to produce a certificate you can confidently hand to a customer. When you encounter a vehicle not in your database, UKVD at £0.15 fills the gap while you add that model to your JSON file for future lookups. The key insight is simple: **let Claude do what it's best at (writing clear, contextualised explanations) and keep the factual data in systems you control**.